Solution: Daily Temperatures

Problem Statement

Given an array of integers `temperatures` representing daily temperatures, your task is to calculate how many days you have to wait until a warmer temperature. If there is no future day for which this is possible, put 0 instead.

Examples

1. - Input: `temperatures = [70, 73, 75, 71, 69, 72, 76, 73]`
   - Output: `[1, 1, 4, 2, 1, 1, 0, 0]`
   - Explanation: The first day's temperature is 70 and the next day's temperature is 73 which is warmer. So for the first day, you only have to wait for 1 day to get a warmer temperature. Hence, the first element in the result array is 1. The same process is followed for the rest of the days.
2. - Input: `temperatures = [73, 72, 71, 70]`
   - Output: `[0, 0, 0, 0]`
   - Explanation: As we can see, the temperature is decreasing every day. So, there is no future day with a warmer temperature. Hence, all the elements in the result array are 0.
3. - Input: `temperatures = [70, 71, 72, 73]`
   - Output: `[1, 1, 1, 0]`
   - Explanation: For the first three days, the next day is warmer. But for the last day, there is no future day with a warmer temperature. Hence, the result array is `[1, 1, 1, 0]`.

Solution

This problem is quite similar to 'Next Greater Element'. We can use a monotonically decreasing stack to find the next higher temperature.

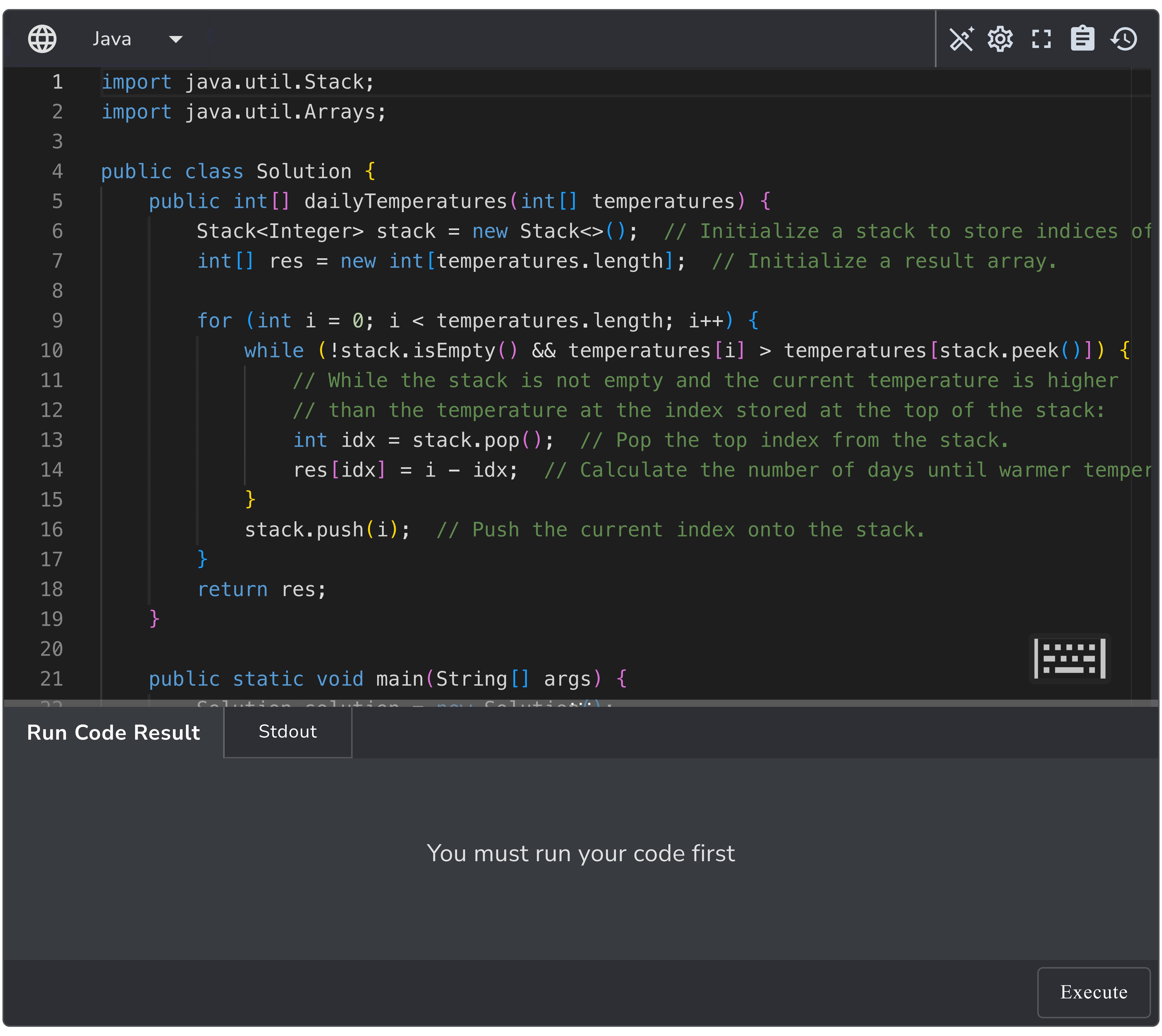
We will use a stack to store the indices of the `temperatures` array. We iterate over the array, and for each temperature, we check whether the current temperature is greater than the temperature at the index on the top of the stack. If it is, we update the corresponding position in the result array and pop the index from the stack.

Algorithm Walkthrough

1. Initialize an empty stack to store the indices of the `temperatures` array. Also, initialize a result array of the same length as `temperatures` with all values set to 0.
2. Iterate over the `temperatures` array. For each temperature:
   - While the stack is not empty and the current temperature is greater than the temperature at the index on the top of the stack, set the value in the result array at the top index of the stack to the difference between the current index and the top index of the stack. Pop the index from the stack.
   - Push the current index onto the stack.
3. Return the result array.

Code

Here is how we can implement this algorithm:



```java
import java.util.Stack;
import java.util.Arrays;

public class Solution {
    public int[] dailyTemperatures(int[] temperatures) {
        Stack<Integer> stack = new Stack<>();  // Initialize a stack to store indices of
        int[] res = new int[temperatures.length];  // Initialize a result array.

        for (int i = 0; i < temperatures.length; i++) {
            while (!stack.isEmpty() && temperatures[i] > temperatures[stack.peek()]) {
                // While the stack is not empty and the current temperature is higher
                // than the temperature at the index stored at the top of the stack:
                int idx = stack.pop();  // Pop the top index from the stack.
                res[idx] = i - idx;  // Calculate the number of days until warmer temper
            }
            stack.push(i);  // Push the current index onto the stack.
        }
        return res;
    }

    public static void main(String[] args) {
```

You must run your code first

Time and Space Complexity

The time complexity of the algorithm is $O(N)$, where N is the size of the temperatures array, since we process each temperature once. The space complexity is also $O(N)$, where N is the size of the temperatures array, due to the extra space used by the stack and the output list.

← Previous
Daily Temperatures (Easy)

Next →
Remove Nodes From Linked List (Easy)

☐ Mark as Completed